Introduction à la Programmation et l'Algorithmique

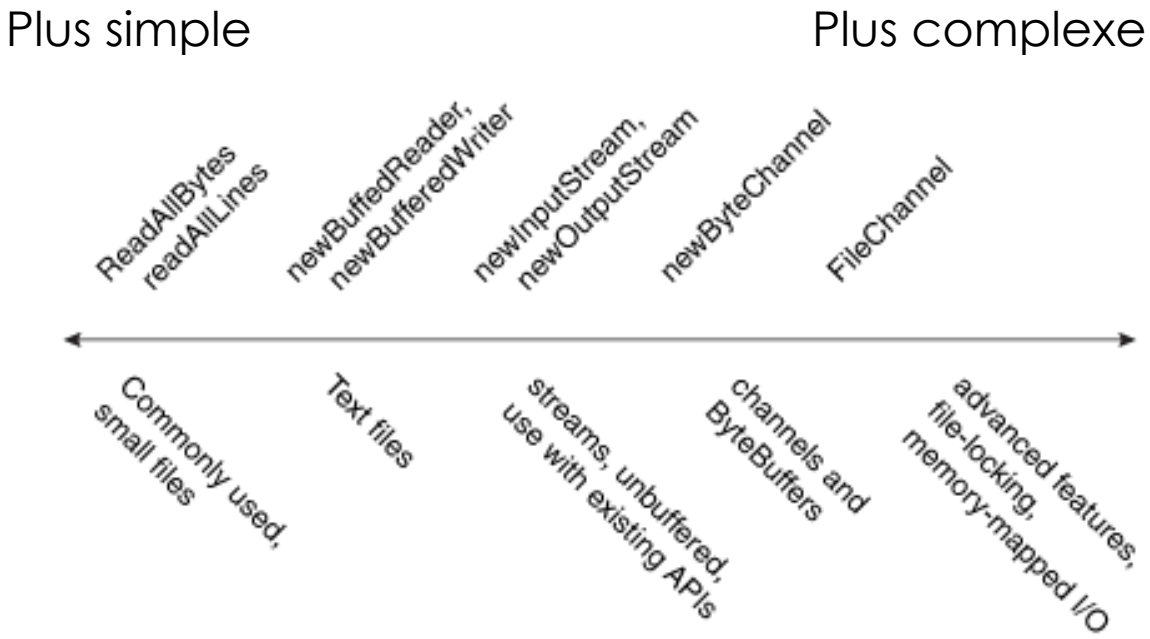
DINO LOPEZ PACHECO

DINO.LOPEZ@UNIV-COTEDAZUR.FR

Les fichiers – NIO.2

Travailler avec des fichiers

- Plusieurs librairies
  - IO
  - NIO/NIO2
- IO
  - Simple, *stream-oriented, blocking*
- NIO
  - Plus complexe, *buffer-oriented, non-blocking*
  - Nouvelles capacités – e.g. support de symlinks



Créer un Path

- Dans NIO, tout commence par la création d'un Path à partir de Paths
  - Pour créer un Path, on utilise la méthode statique get() de Paths
  - java.nio.file.Paths, java.nio.file.Path
- Un Path peut être créé à partir d'un chemin relatif ou absolut
  - Path p = Paths.get("/home/user/test/fichier.txt");
  - Path p = Paths.get("test/fichier.txt");
- Quelques opération avec Path
  - getNameCount() -> nombre d'étiquettes dans le chemin
  - getFileName() –> dernier élément dans le chemin
  - getParent() -> dossier parent (si renseigné lors de la création de Path)
  - getRoot() -> racine (si disponible lors de la création de Path)
  - toAbsolutPath() –> récupérer un Path avec un chemin absolu
  - toRealPath() -> récupérer un Path avec le vrai chemin (symlinks)
  - Plus d'informations à https://docs.oracle.com/javase/8/docs/api/java/nio/file/Path.html

Files : opérations sur un Path

- La classe « Files » permet de retrouver les caractéristiques du Path
  - « Files » contient que des méthodes statiques
- Opérations fréquentes
  - isDirectory() / isRegularFile()
  - isExecutable()
  - isReadable()
  - isWritable()
  - isSymbolicLink()
  - move() / copy()
  - list()
  - size()
- Opérations sur de petits fichiers
  - readAllLines()
  - readAllBytes()
  - write() -> méthode surchargée pour écrire un tableau d'octets ou de lignes de texte dans le Path
- Sur de gros fichiers texte
  - newBufferedReader() / newBufferedWriter()
- Fichiers binaires – style classique, sans buffer
  - newInputStream() / newOutputStream()
- Fichier binaires – avec des buffers
  - newByteChannel()

readAllLines() et readAllBytes()

```
public class TestFiles {
    Path p;
    public TestFiles(String s) {
        p = Paths.get(s);
        if (Files.exists(p)) System.out.println("Fichier existant");
        if (Files.isDirectory(p)) System.out.println("C'est un dossier");
    }
    …
```

```
public void testRAL() {
  try {
    List<String> al = Files.readAllLines(p);
    for(String line : al)
      System.out.println(line);
  } catch(IOException ioe) {…}
}
```

```
public void testRAB() {
  try {
    byte[] b = Files.readAllBytes(p);
    System.out.println(new String(b));
  } catch(IOException ioe) {…}
}
```

Write (tableau byte et liste de String)

```
try {
  byte[] b = ("Hello!").getBytes();
  Files.write(p, b, StandardOpenOption.CREATE, StandardOpenOption.APPEND);
  ArrayList<String> al = new ArrayList<String>();
  al.add("world!");
  al.add("mundo!");
  Path tp = Files.write(p, al, StandardOpenOption.APPEND);
} catch(IOException ioe) {…}
```

BufferedReader (try-with-resources)

- Try-with-resources ferme automatiquement le canal de communication
- La ressource doit être *AutoClosable*

```
try (BufferedReader reader = Files.newBufferedReader(p)) {
  String line = null;
  while ((line = reader.readLine()) != null)
    System.out.println(line);
} catch (IOException ioe) {…}
…
try (BufferedReader reader = Files.newBufferedReader(p)) {
  char[] c = new char[100];
  int n = reader.read(c);
  while (n > 0) {
    System.out.print((new String(c)).substring(0,n));
    n = reader.read(c);
  }
} catch (IOException ioe) {…}
```

BufferedWriter

```
try (BufferedWriter w = Files.newBufferedWriter(p,StandardOpenOption.CREATE,
StandardOpenOption.APPEND)) {
  String line = "Bonjour";
  w.write(line);
  w.write(line);
  w.newLine(); // sinon, pas de retour à la ligne
  w.write(line);
} catch (IOException ioe) {…}
```

Fichiers binaires : InputStream / OutputStream

```
try(InputStream is = Files.newInputStream(p)) {
  byte[] b = new byte[100];
  int n = is.read(b);
  // si on lit moins de 100o
  byte[] res = java.util.Arrays.copyOfRange(b, 0, n);
  System.out.printf("I read %d bytes :%n%s",n,new String(res));
} catch(IOException ioe) {…}


// avec TRUNCATE_EXISTING on vide le fichier complètement
try(OutputStream os = Files.newOutputStream(p,StandardOpenOption.CREATE,
StandardOpenOption.WRITE, StandardOpenOption.TRUNCATE_EXISTING)) {
  byte[] b = ("hello world!").getBytes();
  os.write(b);
} catch(IOException ioe) {…}
```

Channels et ByteBuffer

```
try(SeekableByteChannel sbc = Files.newByteChannel(p,StandardOpenOption.CREATE,
StandardOpenOption.WRITE, StandardOpenOption.READ)) {
  ByteBuffer bf = ByteBuffer.allocate(100); // buffer de 100o
  int n = sbc.read(bf);
  System.out.println("Got " + n + " bytes\nI read:\n" + new String(bf.array()));
  sbc.position(sbc.size()); // on avance à la fin du fichier
  String s = "monde!\n";
  ByteBuffer bf2 = ByteBuffer.allocate(100);
  //bf2 = ByteBuffer.wrap(s.getBytes()); //on construit un buffer autour de ce byte[]
  bf2.put(s.getBytes()); //on met peu de bytes, le compteur position avance avec put()
  bf2.rewind(); // du coup, on revient en debut du buffer
  bf2.limit(s.getBytes().length); //on dit que la fin ce n'est plus 100, mais length
  sbc.write(bf2);
} catch (IOException x) {…}
```

Et pour finir

- Scanner est aussi compatible avec NIO
  - Scanner(Path source)
  - vous pouvez utiliser les hasNext(), nextLine(), nextInteger(), etc de Scanner
- Quelques URL avec de docs (mais n'hésitez pas à rechercher ce dont vous aurez besoin):
  - https://docs.oracle.com/javase/7/docs/api/java/util/Scanner.html
  - https://docs.oracle.com/javase/8/docs/api/java/nio/file/Files.html
  - https://docs.oracle.com/javase/7/docs/api/java/nio/channels/SeekableByteChannel.html
  - https://docs.oracle.com/javase/7/docs/api/java/nio/file/StandardOpenOption.html

- Les import faits pour les démos de pages précédents:

```
import java.nio.file.Paths;
import java.nio.file.Path;
import java.nio.file.Files;
import java.nio.file.LinkOption;
import java.nio.ByteBuffer;
import java.nio.CharBuffer;
import java.nio.file.StandardOpenOption;
import java.nio.channels.SeekableByteChannel;
import java.util.List;
import java.util.ArrayList;
import java.io.IOException;
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.InputStream;
import java.io.OutputStream;
```